

Contents

1	DD14: バッチ・非同期処理	1
1.1	0. 文書情報	1
1.2	1. 対象範囲	1
1.3	2. 収録ロジック・対応章	1
1.4	3. 詳細設計本文	2
1.4.1	3.1 cron 一覧	2
1.4.2	3.2 Queue 一覧	4
1.4.3	3.3 Consumer 共通仕様	5
1.4.4	3.4 幂等性キー管理	5
1.4.5	3.5 実装モジュール構成	5
1.5	4. 関連設計	6
1.6	5. テスト観点	6
1.6.1	5.1 その他観点	6
1.7	6. 未確定事項・確認事項	7

1 DD14: バッチ・非同期処理

1.1 0. 文書情報

項目	内容
文書名	DD14: バッチ・非同期処理
詳細設計 ID	DD14
対象システム	FAQ AI ウィジェット SaaS / メインシステム
関連機能 ID	FR-148 / FR-191 / NFR-501~507 / NFR-801~810 / AC-021 / AC-022
作成日	2026-05-17
版数	v1.0
ステータス	承認済

1.2 1. 対象範囲

種別	ID	名称
cron	10 ジョブ	月次集計 / 月次確定 / トライアル終了判定 / 保持期間処理 / 自動クローズ 等
Queue	5 個	email / announcement-fanout / export / ai-regression / webhook-dlq
Consumer	共通仕様	指数 BO 最大 3 回 / DLQ 4 日 + R2 30 日 / リプレイ
ライブラリ	KV 幂等性キー管理	checkIdempotency / storeIdempotency

1.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 10.11	cron / バッチ全一覧	§ 14.1 参照
§ 14.1.1	月次集計 (UTC 15:00、月末日のみ実行)	usage_metering
§ 14.1.2	月次確定 (JST 02:00、月初 1 日)	billing_invoices
§ 14.1.3	トライアル終了判定 (JST 00:00)	§ 7.10.4 で実装済み
§ 14.1.4	未解決質問保持期間通知 (JST 09:00)	OPEN_INQUIRY_RETENTION_NOTICE
§ 14.1.5	730 日保持期間処理 (JST 02:00)	滞留通知
§ 14.1.6	自動クローズ評価 (5 分間隔)	§ 7.7.4 で実装済み
§ 14.1.7	監査ログ完全性検証 (JST 03:00 daily)	§ 10.8 で実装済み
§ 14.1.8	tombstone バッチ (JST 04:00)	1y/5y/7y 経過
§ 14.1.9	定期削除 (JST 05:00)	保持期間超過の物理削除
§ 14.1.10	D1 容量監視 (毎時)	§ 9.8 で実装済み
§ 14.2	Queue 一覧	5 個

元章	元タイトル	概要
§ 14.3	Consumer 共通仕様	再試行 / DLQ / リプレイ
§ 14.4	冪等性キー管理	KV 24h

1.4 3. 詳細設計本文

1.4.1 3.1 cron 一覧

1.4.1.1 3.1.1 月次集計 (UTC 15:00、月末日のみ実行) 詳細は [DD12_ 利用量計測・課金.md](#) § 3.4 を参照。

1.4.1.2 3.1.2 月次確定 (JST 02:00、月初 1 日) 詳細は [DD12_ 利用量計測・課金.md](#) § 3.5 を参照。

1.4.1.3 3.1.3 トライアル終了判定 (JST 00:00) `contract_owners.trial_ends_at < NOW()` のオーナーをトライアル → free / paid プランへ遷移させる。詳細は基本設計 [../02_ 基本設計/10_ 課金・請求設計.md](#) を参照。

1.4.1.4 3.1.4 未解決質問保持期間通知 (JST 09:00) `case_status='open'` の未解決質問が保持期間に近づいた場合に、管理者ユーザーへ滞留通知を送信する。案件状態は変更しない。

1.4.1.5 3.1.5 730 日保持期間処理 (JST 02:00)

```
export async function processOpenInquiryRetention(env: Env) {
  const cutoff = new Date(Date.now() - 730 * 86400000).toISOString();
  const stuck = await env.DB.prepare(`
    SELECT id, contract_owner_user_id, case_status FROM inquiries
    WHERE case_status = 'open'
    AND created_at <= ?1
    AND valid = 1
  `).bind(cutoff).all(); -- valid=1 で論理削除済みを除く
  for (const i of stuck.results) {
    await enqueueNotification(env, {
      contractOwnerUserId: i.contract_owner_user_id, kind: 'OPEN_INQUIRY_RETENTION_NOTICE',
      refType: 'inquiry', refId: i.id,
    });
    await writeAudit(env, {
      action: 'inquiry.retention_notice', contractOwnerUserId: i.contract_owner_user_id, target:
      retentionClass: 'general',
    });
  }
}
```

1.4.1.6 3.1.6 自動クローズ評価 (5 分間隔) `chat_rooms` を走査し、最終アクティビティからの経過時間に
応じて自動クローズ 6 段階を進める。詳細は基本設計を参照。

1.4.1.7 3.1.7 監査ログ完全性検証 (JST 03:00 daily / 日次全件再計算) メイン側 `audit_logs` は API 操作
ログ専用であり、ハッシュチェーン構造を持たない (`prev_hash` / `current_hash` 列は持たない)。本処理は **運
営者側 `audit_logs` (ハッシュチェーン正本) に対してのみ実行** される。詳細は運営者側設計 § 7 + [DD10_ 監査ロ
グ書込・完全性検証.md](#) § 3.6 を参照。

1.4.1.8 3.1.8 `audit_logs` 保持期間超過削除バッチ (JST 04:00、物理 DELETE 方式) メイン側 `audit_logs`
は「API 操作ログ専用」でありハッシュチェーン正本は運営者側に一本化されている。メイン側ではチェーン破
断を防ぐための tombstone 方式は不要のため、保持期間超過後は物理 DELETE で処理する。

```
export async function purgeExpiredAuditLogs(env: Env) {
  // 一般 (1 年) / 課金 (7 年) / 運営者高権限 (5 年) 経過の audit_logs を物理 DELETE
  await env.DB.prepare(`
    DELETE FROM audit_logs WHERE (
      (retention_class = 'general' AND created_at < ?1) OR
      (retention_class = 'operator_high_priv' AND created_at < ?2) OR

```

```

    (retention_class = 'billing'                                AND created_at < ?3)
  )
  `).bind(
    new Date(Date.now() - 1 * 365 * 86400000).toISOString(),
    new Date(Date.now() - 5 * 365 * 86400000).toISOString(),
    new Date(Date.now() - 7 * 365 * 86400000).toISOString(),
  ).run();
}

```

注: 運営者側 `audit_logs`(ハッシュチェーン正本) では tombstone 方式を維持する (運営者側設計 §7 参照)。本処理はメイン側 D1 限定。

1.4.1.9 3.1.9 定期削除 (JST 05:00)

```

export async function retentionCleanup(env: Env) {
  const now = Date.now();
  await env.DB.batch([
    env.DB.prepare(`DELETE FROM question_logs WHERE created_at < ?1`)
      .bind(new Date(now - 365 * 86400000).toISOString()),
    env.DB.prepare(`DELETE FROM chat_messages WHERE chat_room_id IN (
      SELECT id FROM chat_rooms WHERE closed_at < ?1
    )`)
      .bind(new Date(now - 365 * 86400000).toISOString()),
    env.DB.prepare(`DELETE FROM chat_rooms WHERE closed_at < ?1`)
      .bind(new Date(now - 365 * 86400000).toISOString()),
    env.DB.prepare(`DELETE FROM inquiries WHERE closed_at < ?1 AND case_status = 'closed'`)
      .bind(new Date(now - 365 * 86400000).toISOString()),
    env.DB.prepare(`DELETE FROM notification_logs WHERE created_at < ?1`)
      .bind(new Date(now - 365 * 86400000).toISOString()),
    env.DB.prepare(`DELETE FROM error_logs WHERE occurred_at < ?1`)
      .bind(new Date(now - 180 * 86400000).toISOString()),
    env.DB.prepare(`DELETE FROM access_tokens WHERE expires_at < ?1`)
      .bind(new Date(now - 30 * 86400000).toISOString()),
    env.DB.prepare(`DELETE FROM sessions WHERE revoked_at IS NOT NULL AND revoked_at < ?1`)
      .bind(new Date(now - 30 * 86400000).toISOString()),
    env.DB.prepare(`DELETE FROM inbox_messages WHERE created_at < ?1`)
      .bind(new Date(now - 365 * 86400000).toISOString()),
    // 退会確定: contract_owners.contract_status を deleted_pending → deleted (30 日経過、valid=0)
    // オーナー判定は contract_owners 行存在で行うため、本 UPDATE は contract_owners テーブルを対象とする
    // また、当該オーナーの users 行も valid=0 にする必要があるが、それは下記 hardDeleteLogicalDeleted
    // バッチで論理削除済 users をまとめて物理削除する前段として、別途 withdrawal-finalize 処理 (SCR-02)
    // contract_owners と users の双方を valid=0 に揃える運用とする。
    env.DB.prepare(`
      UPDATE contract_owners SET contract_status = 'deleted', valid = 0, updated_at = ?1
      WHERE contract_status = 'deleted_pending'
      AND user_id IN (SELECT contract_owner_user_id FROM withdrawal_requests
        WHERE scheduled_deletion_at < ?1)
    `)
      .bind(new Date().toISOString()),
  ]);
}

```

1.4.1.10 3.1.10 D1 容量監視 (毎時) D1 の容量を監視し、80% を超えるとアラート (運営者 inbox に high)。詳細は基本設計を参照。

1.4.1.11 3.1.11 論理削除データの 90 日後物理削除バッチ (日次 03:00 JST) 全削除対象テーブル (`users` / `contract_owners` / `project_users` / `projects` / `allowed_domains` / `project_ip_allowlist` / `faqs` / `question_logs` / `inquiries` / `inquiry_contacts` / `chat_rooms`) の `valid` カラムによる論理削除データを、`updated_at` 基準で 90 日経過後に物理削除する。`billing_subscriptions` は除外 (電子帳簿保存法 7 年保持、永久維持)。

処理ステップ:

1. 監査ログ `user.hard_delete_by_batch` / `project.hard_delete_by_batch` を物理 DELETE 前に発行 (`retention_class='general'`、metadata に削除対象 ID と論理削除日時を含む)

2. `contract_owners.data_deletion_mode='anonymize'` の対象オーナーは物理 DELETE 前に匿名化処理 (03_ テーブル設計.md §9 削除モード表に従う)
3. `users` 行を物理 DELETE (ON DELETE CASCADE で関連 `contract_owners / project_users / sessions / access_tokens / terms_agreements / inquiry_contacts` 等が連鎖物理削除)
4. `projects` 行を物理 DELETE (ON DELETE CASCADE で関連 `faqs / inquiries / chat_*` 等が連鎖物理削除)
5. 中間テーブル / 履歴系の独立物理削除 (`allowed_domains / project_ip_allowlist / question_logs` 等で親が既に物理削除されていない孤立行があれば独立 DELETE)

// 物理削除バッチ (疑似コード)

```
async function hardDeleteLogicalDeletedUsers(env, now) {
  const cutoff = new Date(now - 90 * 86400000).toISOString();
  // 1. 監査ログを先行発行。論理削除済オーナー (contract_owners 行が valid=0) は data_deletion_mode を
  const targets = await env.DB.prepare(`
    SELECT u.id AS user_id, co.user_id AS contract_owner_user_id, co.data_deletion_mode
    FROM users u
    LEFT JOIN contract_owners co ON co.user_id = u.id
    WHERE u.valid = 0 AND u.updated_at < ?1
  `).bind(cutoff).all();
  for (const u of targets.results) {
    await env.DB.prepare(`INSERT INTO audit_logs (...) VALUES (...)`).bind(
      'user.hard_delete_by_batch', 'user', u.user_id,
      JSON.stringify({ contractOwnerUserId: u.contract_owner_user_id ?? null }), 'general'
    ).run();
    // 匿名化はオーナーのみ (契約配下データへの永続書換が発生するため)
    if (u.contract_owner_user_id && u.data_deletion_mode === 'anonymize') {
      // 2. 匿名化処理 (03_ テーブル設計 §9 に従う)
      await anonymizeContractOwnerData(env, u.contract_owner_user_id);
    }
  }
  // 3. users 物理 DELETE (CASCADE で contract_owners / project_users / sessions / access_tokens
  await env.DB.prepare(`DELETE FROM users WHERE valid = 0 AND updated_at < ?1`).bind(cutoff).run();
  // 3a. 念のため、users が valid=1 で残ったまま contract_owners 単独が valid=0 のケース (将来の運用変
  await env.DB.prepare(`DELETE FROM contract_owners WHERE valid = 0 AND updated_at < ?1`).bind(cutoff).run();
  // 3b. project_users 単独論理削除 (プロジェクト削除起因でメンバー by メンバーに valid=0 が発生したケー
  await env.DB.prepare(`DELETE FROM project_users WHERE valid = 0 AND updated_at < ?1`).bind(cutoff).run();
  // 4. projects 物理 DELETE (CASCADE)
  await env.DB.prepare(`DELETE FROM projects WHERE valid = 0 AND updated_at < ?1`).bind(cutoff).run();
  // 5. 孤立中間テーブル行の物理 DELETE
  await env.DB.prepare(`DELETE FROM allowed_domains WHERE valid = 0 AND updated_at < ?1`).bind(cutoff).run();
  await env.DB.prepare(`DELETE FROM project_ip_allowlist WHERE valid = 0 AND updated_at < ?1`).bind(cutoff).run();
  await env.DB.prepare(`DELETE FROM faqs WHERE valid = 0 AND updated_at < ?1`).bind(cutoff).run();
  await env.DB.prepare(`DELETE FROM question_logs WHERE valid = 0 AND updated_at < ?1`).bind(cutoff).run();
  await env.DB.prepare(`DELETE FROM inquiries WHERE valid = 0 AND updated_at < ?1`).bind(cutoff).run();
  await env.DB.prepare(`DELETE FROM inquiry_contacts WHERE valid = 0 AND updated_at < ?1`).bind(cutoff).run();
  await env.DB.prepare(`DELETE FROM chat_rooms WHERE valid = 0 AND updated_at < ?1`).bind(cutoff).run();
  // billing_subscriptions は対象外 (永久保持)
}
```

注意:

- `users` の物理 DELETE は ON DELETE CASCADE で広範囲が連鎖する (`contract_owners / project_users / sessions / access_tokens / terms_agreements` 等)。そのため `users` を最初に処理し、その後の独立 DELETE は孤立行を拾うのみとなる
- 監査ログ自体は tombstone 方式 (`audit_logs.tombstone`) で保持されるため、物理削除バッチは本テーブルには適用しない
- 90 日カットオフは `updated_at` 基準。論理削除以降に `updated_at` が変動するイベント (例: 復元処理) が発生した場合は再カウントとなる (復元から 90 日後の物理削除)。MVP では復元 UI は持たず、運営者 IF #4 経由のみ。

1.4.2 3.2 Queue 一覧

Queue 名	用途	producer	consumer	最大同時	再試行	DLQ
email-queue	メール送信	enqueueEmail()	consumers/email.ts	10	3 回（指数 BO）	webhook-dlq
announcement-fanout-queue	お知らせ配信	IF #7 受信	consumers/fanout.ts	5	3 回	同上
export-queue	データエクスポート	data/export	consumers/export.ts	3	2 回	同上
ai-regression-queue	AI 回帰テスト	IF (v2.1)	consumers/ai-regression.ts	1	1 回	同上
webhook-dlq	DLQ 退避	上記 consumer 失敗時	consumers/dlq.ts (運営者通知)	-	-	-

1.4.3 3.3 Consumer 共通仕様

1.4.3.1 3.3.1 指数 BO + 最大 3 回再試行

```
catch (e) {
  if (msg.attempts >= MAX_RETRY) {
    await dlqEnqueue(env, msg.body, e);
    msg.ack();
  } else {
    const delay = Math.min(60 * Math.pow(2, msg.attempts), 600); // 60s, 120s, 240s, ..., max
    msg.retry({ delaySeconds: delay });
  }
}
```

1.4.3.2 3.3.2 DLQ 4 日 + R2 退避 30 日 DLQ Queue は 4 日保持。webhook-dlq consumer が R2 にペイロードを 30 日退避し、運営者へ critical アラート。

```
export async function dlqConsume(batch: MessageBatch<DlqEntry>, env: Env) {
  for (const msg of batch.messages) {
    const key = `dlq/${msg.body.queue}/${msg.body.messageId}.json`;
    await env.R2_DLQ.put(key, JSON.stringify(msg.body), {
      customMetadata: { expiresAt: String(Date.now() + 30 * 86400000) },
    });
    await enqueueServiceAlert(env, {
      level: 'high', channel: 'ops_high',
      message: `DLQ 滞留: queue=${msg.body.queue}, msgId=${msg.body.messageId}, R2 退避 =${key}`
    });
    msg.ack();
  }
}
```

1.4.3.3 3.3.3 リプレイ機構 DLQ R2 退避から手動でリプレイ可能。scripts/dlq-replay.ts で R2 から読み込み → 元 Queue に再投入。30 日以内に限る。

1.4.4 3.4 冪等性キー管理

```
// KV: idempotency:{key} (TTL 24h)
export async function checkIdempotency(env: Env, key: string): Promise<{ cached: any } | null> {
  const v = await env.KV_DEDUP.get(`idempotency:${key}`, 'json');
  return v ? { cached: v } : null;
}
export async function storeIdempotency(env: Env, key: string, result: any) {
  await env.KV_DEDUP.put(`idempotency:${key}`, JSON.stringify(result), { expirationTtl: 86400 })
}
```

1.4.5 3.5 実装モジュール構成

```
worker-cron/src/
└─ index.ts # scheduled() ハンドラ + cron 分岐
```

```

├─ jobs/
│   ├── monthly-aggregate.ts          # UTC 15:00 (月末日)
│   ├── monthly-finalize.ts          # JST 02:00 (月初 1 日)
│   ├── trial-end-check.ts           # JST 00:00 (毎日)
│   ├── open-inquiry-retention-notice.ts # JST 09:00 (毎日)
│   ├── open-inquiry-retention.ts    # JST 02:00 (毎日)
│   ├── auto-close-evaluation.ts     # */5 * * * * (5 分間隔)
│   ├── audit-chain-verify.ts        # JST 03:00 (日次全件再計算)
│   ├── tombstone-batch.ts           # JST 04:00 (毎日)
│   ├── retention-cleanup.ts         # JST 05:00 (毎日)
│   └── d1-capacity-check.ts         # 0 * * * * (毎時)
└─ lib/

worker-queue-consumer/src/
├─ index.ts                          # Queue Consumer 共通エントリ
├─ consumers/
│   ├── email.ts                     # Resend 送信
│   ├── fanout.ts                    # お知らせ fan-out
│   ├── export.ts                    # データエクスポート生成
│   ├── ai-regression.ts             # AI 回帰テスト実行
│   └── dlq.ts                       # DLQ 退避 (R2 退避 + 監視通知)
└─ lib/
    └─ retry.ts                      # 指数バックオフ + 最大 3 回

```

1.5 4. 関連設計

種別	参照先
要件	../01_要件定義/index.md
基本設計	../02_基本設計/index.md
API 設計	../02_基本設計/02_API 設計.md
課金・請求設計	../02_基本設計/10_課金・請求設計.md
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md
関連 DD	DD05_inquiry_code 採番・未解決質問.md / DD07_通知ロジック.md / DD10_監査ログ書込・完全性検証.md / DD12_利用量計測・課金.md

1.6 5. テスト観点

AC ID	テスト ID	テスト方式	テストファイル
AC-021	it-notification-001	Integration	workers/queue-consumer/test/integration/notification.test.ts
AC-022	e2e-announcements-001	E2E	apps/admin/e2e/announcements/inbox.spec.ts
AC-031	it-audit-chain-001	Daily cron + Manual	workers/cron/test/integration/audit-chain-verify.test.ts
AC-032	it-tombstone-001	Integration	workers/cron/test/integration/tombstone.test.ts

1.6.1 5.1 その他観点

観点	内容
単体	retentionCleanup 各テーブルの DELETE 条件 / tombstoneBatch の SET NULL 範囲
結合	cron 連鎖 (月次集計 → 月次確定 → IF #10 → 通知 → 監査ログ)
異常系	Consumer 3 回失敗 → DLQ → R2 退避 → 運営者 critical / リプレイで再処理成功

観点	内容
境界値	保持期間 364 日 / 365 日 / 366 日 / 730 日経過時の挙動
冪等性	同じ Idempotency-Key で重複処理しない / 同月の月次確定再実行で二重請求なし
性能	5 分間隔 cron が 5 分以内に完了 / 日次 cron が深夜帯（02:00-05:00）に収まる

1.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済